

FULL STACK DEVELOPMENT

FASE 5 | AULA 02 -

ENTENDO A ARQUITETURA NA CLEAN ARCHITECTURE

SUMÁRIO

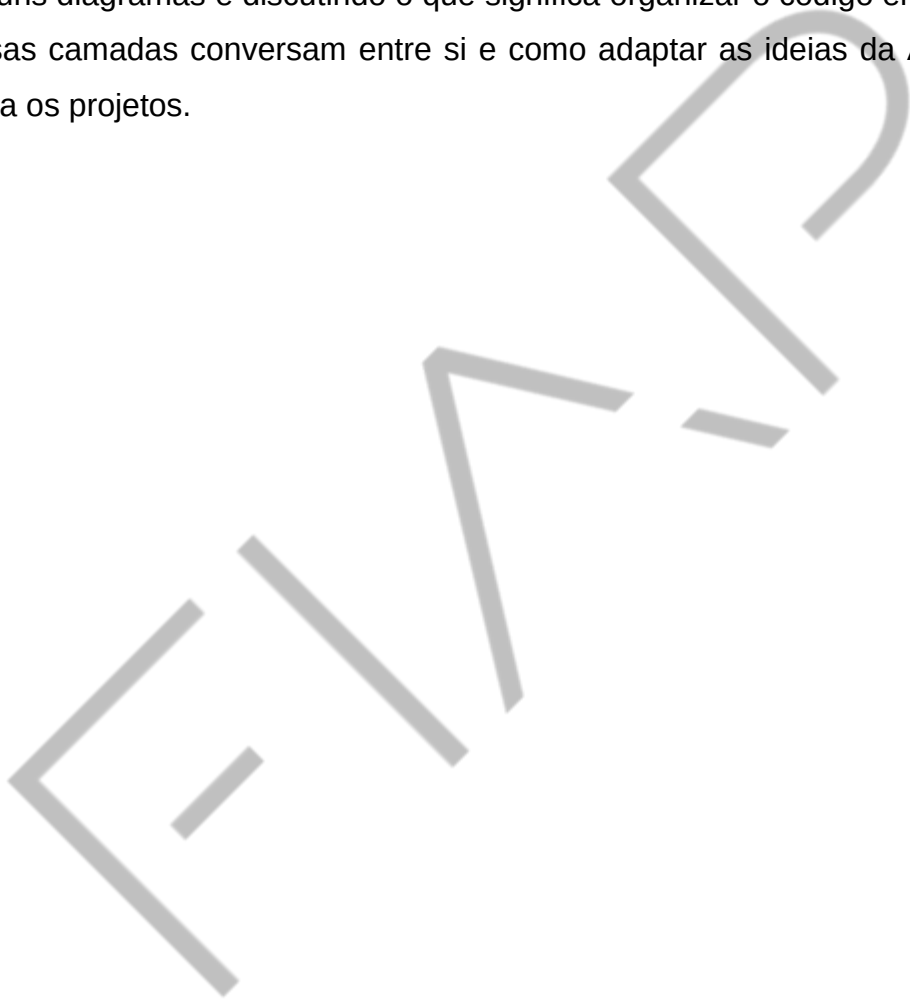
O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	11
REFERÊNCIAS.....	12

ENTENDENDO

O QUE VEM POR AÍ?

A Arquitetura Limpa é mais do que uma solução de arquitetura: ela define um conjunto de regras aplicáveis em todos os níveis da solução, sendo eles a codificação, organização de unidades de código e definição de camadas.

Vamos entender um pouco sobre as ideias da Arquitetura Limpa analisando alguns diagramas e discutindo o que significa organizar o código em camadas, como essas camadas conversam entre si e como adaptar as ideias da Arquitetura Limpa para os projetos.



HANDS ON

Veremos uma discussão aprofundada sobre os componentes da Arquitetura Hexagonal e Limpa nos vídeos a seguir, com o entendimento do papel das entidades dentro do software, como a separação de responsabilidades é reforçada dentro de cada uma das camadas e como a Arquitetura Limpa e a Arquitetura Hexagonal definem pontos comuns cruciais para um bom projeto de software desde a sua base.



SAIBA MAIS

Quando pensamos em arquitetura de software, sempre vem à cabeça uma série de documentos, diagramas, estruturas e coisas similares, que definem o que deve ser feito de forma descritiva e bem definida. Estes itens são importantes para a comunicação do que deve ser feito e como, documentando as decisões tomadas e os caminhos do projeto e do software, baseando as tomadas de decisões futuras.

Ao estudarmos sobre os diagramas usados para desenhar os projetos de arquitetura limpa, dois tipos de desenho são mais comuns e sempre são usados como base para o desenho da arquitetura do projeto.

Um dos diagramas é o da Arquitetura Hexagonal, e o outro é o definido no livro “Arquitetura Limpa”, de Robert Martin. Os dois são muito usados como referência e citados em desenhos de projetos como “baseado na ideia de arquitetura hexagonal” ou “baseado nas ideias de arquitetura limpa”.

É importante deixar claro que elas não são a mesma coisa e que muitas das ideias e conceitos da Arquitetura Hexagonal são semelhantes às das definidas na Arquitetura Limpa. Vamos começar a nossa análise dos elementos da Arquitetura Limpa baseando-se nas ideias que encontramos nos diagramas da Arquitetura Hexagonal.

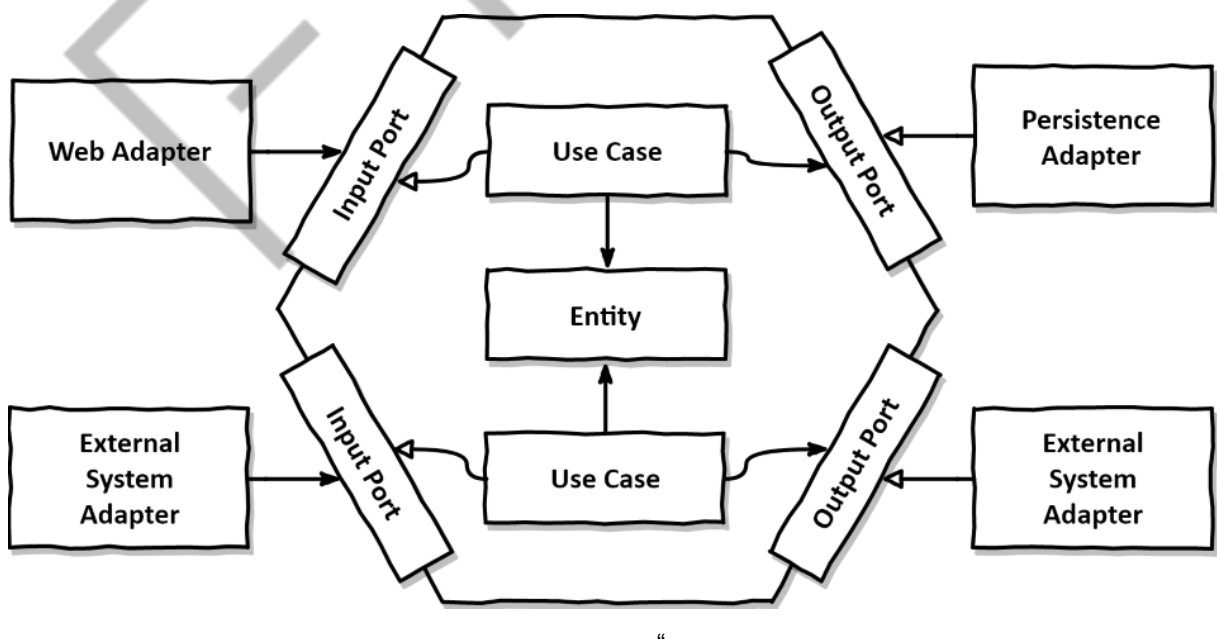


Figura 1 – Diagrama

Fonte: <https://reflectoring.io/spring-hexagonal/> (2019)

O desenho da figura 1 – “Diagrama” é referente à uma aplicação da Arquitetura Hexagonal. Nele, vemos algumas ideias iniciais que são compartilhadas entre a Arquitetura Hexagonal e a Arquitetura Limpa, e a primeira ideia compartilhada entre os dois é a definição de que no centro do desenho temos a camada de Entidades (Entity). As Entidades são os elementos de negócio do sistema: uma “Venda”, um(a) “Aluno(a)”, uma “Ordem de Pagamento”. As entidades modelam os atores de um campo de conhecimento, e modelam de acordo com a abstração necessária para o funcionamento do software.

Essa abstração sempre deve ser feita de acordo com a necessidade do software e nunca devemos incluir informações que não são essenciais para o software; essa abstração é fruto da análise das necessidades, não da reflexão direta da vida real dentro do software. Esta camada de Entidades é **central** no processo, sendo o núcleo da arquitetura, onde colocamos não apenas os dados, mas também as regras e restrições relativas a cada elemento, como regras de consistência dos elementos (não pode existir uma “Matrícula” sem um “Estudante”, ou não pode existir uma “Venda” sem um “Cliente”).

Então, lembrando do que falamos anteriormente sobre o SOLID, cada elemento precisa ser restrito à sua responsabilidade, e implementar apenas o necessário à modelagem de um elemento. Muitas vezes podemos até isolar conceitos simples do sistema como Entidades, de forma a ajudar na construção.

Um exemplo de Entidade, por exemplo, pode ser uma “Pessoa”, como vemos abaixo:

```
class CNPJ {
  value: string;

  constructor(cnpj: string) {
    if (!CNPJ.validar(cnpj)) {
      throw new Error("CNPJ invalido");
    }
    this.value = cnpj;
  }

  static validar(valor: string): boolean {
    // validar o cnpj
    return true;
  }
}
```

```
}  
}  
  
class CPF {  
  value: string;  
  
  constructor(cpf: string) {  
    if (!CPF.validar(cpf)) {  
      throw new Error("CPF invalido");  
    }  
    this.value = cpf;  
  }  
  
  static validar(valor: string): boolean {  
    // validar o cpf  
    return true;  
  }  
}  
  
type Pessoa = {  
  nome: string;  
}  
  
type PessoaFisica = Pessoa & {  
  nascimento: Date;  
  cpf: CPF;  
}  
  
type PessoaJuridica = Pessoa & {  
  cnpj: CNPJ;  
  data_abertura: Date;  
}
```

Temos uma Entidade de nome “Pessoa”, que aqui tem duas Entidades: “PessoaFisica” e “PessoaJuridica”. Ambas têm campos de tipos nativos da linguagem usada (string, Date) e de tipos que tem classes personalizadas (CPF e CNPJ), que podem posteriormente ser validados de acordo com as regras de cada campo.

Aqui vemos outra possibilidade, usar Entidades como componentes de outras, porque isso é parte da responsabilidade dessa camada: modelar os elementos e suas conexões. Aqui, podemos definir que uma “Movimentação Financeira” precisa de uma “Conta Origem”, uma “Conta Destino” e um “Valor”, e que “Conta Origem” e “Conta Destino” são do tipo “Conta”.

Neste momento usamos os nomes de **Entidades do Negócio**, nos baseando na ideia de linguagem ubíqua definida inicialmente no livro “Domain-Driven Design”, de Eric Ryans; isso porque vamos falar na linguagem da Regra de negócio, ou do Domínio, e não na linguagem da implementação (Typescript, C#, Java, etc...), e a linguagem da regra de negócio deve estar vinculada com os termos de linguagem usados pelo(a) cliente, porque estamos modelando os elementos da regra de negócio que o software implementa, e não os detalhes técnicos usados para implementar.

Este isolamento também permite que a regra seja **testável**. Podemos escrever testes para verificarmos se o núcleo do sistema funciona, e definir nos testes um pouco das regras de negócio implícitas, como validação de campos, validação de dados, dependências diretas e indiretas entre os elementos.

Casos de Uso

Agora veremos a camada de **casos de uso**, onde estão implementadas as regras de negócio a serem implementadas pelo software. Aqui temos, um a um, todos os casos de uso que foram identificados no momento da análise das necessidades, além das regras complexas que coordenam diversas Entidades e garantem que os casos de uso funcionem como esperado. Essas regras são isoladas numa camada específica, fora da camada de Entidades e desconectada da camada de interface com o “mundo exterior”.

Um exemplo de Caso de Uso implementado, de acordo com o código anterior, poderia ser o de “Cadastrar uma PessoaFísica”

```
class CadastrarClientePessoaFisicaUseCase {  
  /* recebe os dados de uma pessoa fisica, verifica se estao corretos */  
  pessoaFisica: PessoaFisica;  
  
  constructor(nome: string, cpf: string, nascimento: Date) {  
    this.pessoaFisica.nome = nome;  
    this.pessoaFisica.nascimento = nascimento;  
    this.pessoaFisica.cpf = new CPF(cpf);  
  }  
}
```

Aqui estamos recebendo os dados do componente que utiliza este caso de uso e com estes dados criamos um objeto do tipo PessoaFisica, validamos e, caso esteja tudo certo, adicionamos ao Repositório.

Usando nosso outro exemplo anterior, nessa camada nós implementamos a ação “Efetuar Movimentação Financeira”, que recebe os dados das contas de origem e destino, o valor da transação e valida se tudo está funcionando como esperado.

Perceba que, até este momento, não falamos diretamente sobre o armazenamento de dados, as rotas de API e nada disso; no máximo, recebemos os dados para execução do caso de uso, vindo de uma camada exterior, porque esta é uma das diferenças da Arquitetura Limpa para os outros tipos de arquitetura: a implementação tem que refletir a regra de negócio e seus elementos, e tudo o que não está relacionado a isso é tratado como externo à solução, com a implementação desses itens feita em camadas externas.

Como aprendemos, o código usado para implementar o caso de uso só pode usar outros casos de uso ou os componentes da camada de Entidades. Os casos de uso podem até ser organizados em pacotes ou algo parecido, nas linguagens de programação de implementação, mas as regras de acesso devem que ser mantidas.

O objetivo, com os casos de uso e as Entidades, é isolar as regras de negócio do resto do sistema. E para comunicar “para fora”, como faremos?

Portas e Adaptadores

A implementação das interfaces de comunicação com o mundo exterior acontece para atender duas necessidades: permitir que os dados sejam trafegados de forma padronizada e garantir que as requisições sejam atendidas de forma adequada. Se pensarmos na realidade, essas interfaces se parecem com portas de comunicação, que são adequadas a uma ou outra necessidade, e definem as regras de comunicação. Uma porta USB, por exemplo, serve para trafegar dados entre dois dispositivos, e tem padrões que definem como a comunicação deve ser feita, desde o desenho do plugue até os padrões lógicos de troca de dados.

Na implementação do código usando Arquitetura Hexagonal, temos o conceito de Ports, que fazem um trabalho semelhante, mas são divididas em Input Port, para informar os dados necessários para o componente na camada de caso de uso, e Output Ports, para definir como as respostas serão entregues.

Outra camada detalhada na Arquitetura Hexagonal são os Adaptadores (Adapters), que são componentes que servem como adaptadores de interfaces e dados, para que a comunicação com um elemento externo ocorra de forma adequada à necessidade do funcionamento do software. Usualmente, essa camada serve para isolar e abstrair, para as camadas internas do software, o funcionamento de um elemento externo (como um banco de dados, uma conexão de API, ou o barramento de API que nosso software implementa), tratando dos detalhes de comunicação e fazendo a acomodação dos formatos de dados; por exemplo, se um dado vem de um banco de dados e precisa ser informado via Json para a Input Port, o Adapter é que faz este trabalho.

E a Arquitetura Limpa?

A Arquitetura Limpa implementa ideias semelhantes, mas com algumas diferenças fundamentais.

A primeira diferença é que as camadas externas a de casos de uso são divididas em duas camadas, uma com os componentes de interface e adaptadores, que servem como uma ligação entre o mundo exterior e o núcleo do software, e uma mais externa que serve para implementar as interfaces com o mundo exterior, como os bancos de dados e as APIs. Nas próximas aulas, entenderemos melhor como cada uma dessas camadas funciona e como elas se relacionam.

O QUE VOCÊ VIU NESTA AULA?

Revisamos o conceito de Arquitetura Hexagonal, lembrando das camadas de **Entidades**, de **Casos de Uso**, das **Portas e Adaptadores**.

No centro do diagrama temos as Entidades e Casos de Uso, que implementam as Regras de Negócio que o software precisa automatizar ou implementar. As Entidades definem os elementos que compõem a Regra de Negócio, como uma “Venda”, um(a) “Cliente”, um(a) “Vendedor(a)” e um “Item de Venda”. Os casos de uso implementam as atividades que o software deve fazer, como “Iniciar” uma venda e “Receber” item de venda. Assim, os casos de uso e as Entidades podem ser pensados como um grupo de componentes que implementam **o que** o software faz e **como** o software funciona.

Para se comunicar com o mundo exterior, que é caracterizado pelos elementos do software que não fazem parte da Regra de Negócio, temos as **Portas** e **Adaptadores**, que funcionam como interfaces para a comunicação entre as camadas interiores e exteriores. Os Adaptadores permitem que o software se comunique com serviços externos (como Repositórios de Dados e APIs) de forma transparente para o software, adaptando os dados como necessário, e as Portas servem para padronizar o acesso e coordenar a execução das requisições ao software.

Por fim, também vimos que a Arquitetura Limpa implementa uma estrutura de camadas com mais elementos e mais regras, que permitem uma melhor organização do código, e entendemos a razão da regra que define que apenas elementos das camadas exteriores podem instanciar os elementos das camadas interiores.

O que achou da aula? Nos conte no Discord!

REFERÊNCIAS

EVANS, E. **Domain-Driven Design: Tackling Complexity in the Heart of Software**. [s.l.]: Addison-Wesley Professional, 2003.

MARTIN, R. **Arquitetura Limpa**. Rio de Janeiro: Alta Books Editora, 2019.

MARTIN, R. **Clean Code: A Handbook of Agile Software Craftsmanship**. [s.l.]: Pearson, 2008.

EVANS

PALAVRAS-CHAVE

Clean Code. Arquitetura Limpa. Arquitetura Hexagonal.

EXEMPLO

POS TECH